Name: _____
Surname: _____
Date: _____

Multiple Choice (10 points each)

Instructions

Circle the correct answer for each question, as each question has **only one** correct answer. Provide reasoning for your answer in the space next to each question.
A correct answer always awards full points (+10 pts).
An incorrect answer with sensible reasoning awards half points (+5 pts).
An incorrect answer with faulty or no reasoning subtracts points (-3 pts).
No answer gives zero points (neither adds nor subtracts points).
Assume each code cell is **run in isolation**, meaning no other code has been run before it.

Questions

1. After we run the code below, what will be the value of "x"?

```
s = "1.5 to the power of 2 is 2.25"
x = 10 * float(s[-4:])
```

   (a) `x = 15.0`

   (b) `x = 22.5`

   (c) The code will raise an error.

2. After we run the code below, what will be the value of "x"?

```
s = "1.5 to the power of 2 is 2.25"
count = 0
for x in s:
    if x in "123456789":
        count += 1
print(x)
```

   (a) `5`

   (b) `6`

   (c) The code will raise an error.

3. Which statement is **FALSE** regarding keyword arguments in Python?

   (a) Keyword arguments are always required.

   (b) You are not allowed to put keyword arguments before positional arguments.

   (c) Keyword arguments are used to set default parameter values.

4. Consider the following block of code. What will be the console's output?

```
def exercise(x: list) -> tuple:
    # Reverse the list
    rev = x[::-1]
    return tuple(rev)

result = exercise("hola")
```

   (a) Python will print "aloh".

   (b) The code runs fine but the user won't see any output.

   (c) The code will raise an error.

5. Which one of these statements is **FALSE** about tuples?

   (a) Tuples are ordered and can be indexed.

   (b) Tuples are mutable.

   (c) Tuples can contain duplicated values.

6. After we run the code below, what will be the value of "numbers_new"?

```
def duplicate_list_elements(ls: list) -> list:
    ls_new = []
    while len(ls) > 0:
        x = ls[0]
        ls_new.extend([x, x])
        ls.remove(x)
    return ls_new

numbers = [1, 2, 3]
numbers_new = duplicate_list_elements(numbers)
```

   (a) `numbers_new = []`

   (b) `numbers_new = [1, 2, 3]`

   (c) `numbers_new = [1, 1, 2, 2, 3, 3]`

Answers: 1-b, 2-a, 3-a, 4-b, 5-b, 6-c

Pseudo-Code

Instructions

In the following exercises, you will be asked to write Python functions by hand. Please follow these guidelines:

- Any build-in function you need is in the cheat-sheet provided.
- Clearly indicate each level of indentation using a right arrow (⇒).
- Ensure your code works correctly for any possible example.
- If your code has errors, it will be evaluated based on the provided docstring, comments, and explanations.

Full points will be awarded for code that performs as expected under all conditions.

7. Approximating e (20 points)

The mathematical constant e, approximately equal to 2.71828, is fundamental in many areas of mathematics. It can be defined in various ways, two of which are:

1. **Limit Definition**:
$$e = \lim_{n \to N} \left(1 + \frac{1}{n}\right)^n$$
2. **Series Expansion** (Maclaurin Series for e^x at $x = 1$):
$$e = \sum_{n=0}^{N} \frac{1}{n!} = 1 + 1 + \frac{1}{2!} + \frac{1}{3!} + \frac{1}{4!} + \dots$$

Where $N \to \infty$. Your task is to implement two functions:

- `e_limit(n)`: This computes an approximation of e using the limit definition. *(5 points)*
- `e_series_expansion(n)`: This computes an approximation using the series expansion. *(15 points)*

Both functions will utilize the argument `n` as the upper limit for approximation (N in the equations above).

Answer:

We need two functions, `e_limit(n)` and `e_series_expansion(n)`. Both receive the integer `n` and return a float, the approximation of e.

Limit definition. This one is a direct translation of the formula, and it does not need a loop: `return (1 + 1/n) ** n`. We need parentheses around `1 + 1/n`, and `n` must be greater than 0. The larger `n` is, the better the approximation. *(+5 points)*

Series expansion. We need to add the terms $1/n!$ for every n from 0 up to N (included). We initialize `total = 0`, and loop with `for i in range(n + 1)`, where the "+1" is needed to include the last term. *(+4 points)*

We do not have a factorial function, so we compute it inside the same loop. We initialize `factorial = 1` before the loop. In each iteration, if `i` is greater than 0 we update `factorial = factorial * i`. This works because $i! = i \cdot (i-1)!$, and because $0! = 1$. *(+6 points)*

Then, in each iteration we add the term to the total with `total += 1 / factorial`. After the loop, we return `total`. *(+5 points)*

Comment: We could also compute each factorial with a second (nested) loop. It gives the same result, but it is slower.

(Optional, code)

```python
def e_limit(n: int) -> float:
    return (1 + 1/n) ** n


def e_series_expansion(n: int) -> float:
    total = 0
    factorial = 1
    for i in range(n + 1):
        # Update the factorial (0! = 1, so we skip i = 0)
        if i > 0:
            factorial = factorial * i
        total += 1 / factorial
    return total
```

8. Sports Participation (20 points)

Create a Python function named `sport_participation(football, basketball)` that takes two parameters:

- `football`: A list containing names of students who participated in football.
- `basketball`: A list containing names of students who participated in basketball.

The function must output a dictionary with the following pairs of keys and values:

- `"num_football"`: Number of students participating in football.
- `"num_basketball"`: Number of students participating in basketball.
- `"overlap"`: The list of students participating in both sports.
- `"num_overlap"`: Number of students participating in both sports.
- `"is_subset"`: Boolean. Is `True` when one of the lists is a subset of the other, and `False` otherwise.

Answer:

The function receives two lists of names, which can contain repeated names, and it returns a dictionary with five keys. We start by initializing an empty dictionary, `out_dict = {}`, and we add each key later with `out_dict[key] = value`. *(+3 points)*

In order to avoid repeated names, we first turn both lists into sets with `set()`. *(+3 points)*

The number of students in each sport is the length of its set, `len()`, which we store in `"num_football"` and `"num_basketball"`. *(+3 points)*

The students who play both sports are the intersection of both sets. We store it in the key `"overlap"` turned back into a list with `list()`, as the exercise asks for a list, and we store its length in `"num_overlap"`. *(+5 points)*

A set is a subset of the other when all its members are also members of the other one. This means that the overlap is equal to the smaller set, i.e. one of the two differences is empty. We can check it with `len(set_football.difference(set_basketball)) == 0` (football is a subset of basketball), or the same with the sets swapped. Using the lengths, `num_overlap` equal to `num_football` or to `num_basketball`, is also valid. We combine both checks with `or`, and store the boolean in `"is_subset"`. Finally, we return the dictionary. *(+6 points)*

(Optional, code)

```python
def sport_participation(football: list, basketball: list) -> dict:
    out_dict = {}
    # Remove repeated names
    set_football = set(football)
    set_basketball = set(basketball)
    # Number of students in each sport
    out_dict["num_football"] = len(set_football)
    out_dict["num_basketball"] = len(set_basketball)
    # Students in both sports
    set_overlap = set_football.intersection(set_basketball)
    out_dict["overlap"] = list(set_overlap)
    out_dict["num_overlap"] = len(set_overlap)
    # Is one of the groups included in the other?
```

```
    football_in_basketball = len(set_football.difference(set_basketball)) ==
    0
    basketball_in_football = len(set_basketball.difference(set_football)) ==
    0
    out_dict["is_subset"] = football_in_basketball or basketball_in_football
    return out_dict
```

Cheatsheet

Given a number named `x`:

- `x % y` returns the remainder of the division $\frac{x}{y}$.
- `x // y` returns the quotient of the division $\frac{x}{y}$.
- `abs(x)` returns the absolute value of `x`.

Given a string named `s`:

- `int(s)` returns an integer. Fails if `s` has invalid characters.
- `float(s)` returns a floating number. Fails if `s` has invalid characters.
- `s.split(" ")` split a string into a list where each word is a list item.
- `s.count(<pattern>)` returns the number of non-overlapping occurrences of `<pattern>`.
- `s.find(<pattern>, <start>)` finds the first occurrence of the specified `<pattern>`, starting from index `<start>`. Returns `-1` if the pattern is not found.

Given a list or tuple named `ls`:

- `len(ls)` returns the number of elements in that list.
- `sum(ls)` returns the sum of the elements of that list.
- `sorted(ls)` returns a sorted version of that list.
- `tuple(ls)` / `set(ls)` returns a tuple / set using the elements of that list.
- `", ".join(ls)` returns a string with the elements of the list separated by commas.

Given a set named `a`:

- `a.add(x)` adds the element `x` to the set `a`.
- `a.union(b)` returns the union of set `a` with another set `b`.
- `a.intersection(b)` returns the intersection of set `a` with another set `b`.
- `a.difference(b)` returns the difference of set `a` with another set `b`.

Given a dictionary named `d`:

- `list(d.keys())` returns a list with the keys of `d`.
- `list(d.values())` returns a list with the values of `d`.
- `list(d.items())` returns a list with the tuples (key, value) of `d`.

Some useful build-in functions:

- `type(x)` returns the type of `x` (int, float, list...).
- `range(start, stop)` generates numbers from `start` until `stop`, not including `stop`.